

FOUNDATIONS OF ARTIFICIAL INTELLIGENCE

BSc Data Science & AI - Semester 2

COMPREHENSIVE REVISION NOTE 2

Intelligent Agents & Knowledge Representation

TOPICS COVERED IN THIS REVISION NOTE

1. Deep Dive into Intelligent Agents
2. Types of Agents (Simple Reflex, Model-Based, Goal-Based, Utility-Based, Learning)
3. Agent Environment Types (PEAS Framework)
4. Why Search Algorithms Are Not Enough
5. Introduction to Knowledge Representation
6. Knowledge-Based Agents Architecture
7. The Knowledge Base and Inference Engine
8. Types of Knowledge Representation
9. Symbolic AI and Its Importance
10. Reasoning: Deductive, Inductive, and Abductive

SECTION 1: INTELLIGENT AGENTS - A COMPREHENSIVE VIEW

1.1 Understanding the Agent Concept

An **agent** is anything that can be viewed as perceiving its environment through sensors and acting upon that environment through actuators. This definition is broad enough to include humans, robots, software programs, and even thermostats. The key insight is that agents are situated in an environment and interact with it continuously.

Think of yourself as an intelligent agent: your eyes, ears, and other sensory organs are your sensors that perceive the environment. Your brain processes this information, creates an internal representation of the world, reasons about it, and decides on actions. Your hands, legs, and voice are your actuators that carry out actions in the environment. This perception-cognition-action cycle continues throughout your life.

The concept of agents is central to AI because it provides a framework for understanding how intelligent systems should be designed. Rather than focusing on isolated capabilities (like playing chess or recognizing faces), the agent perspective emphasizes the complete system that must perceive, reason, and act in a coherent way.

■ KEY COMPONENTS OF AN INTELLIGENT AGENT

Sensors: Devices or mechanisms that receive input from the environment. For a robot, these might include cameras, microphones, touch sensors, and GPS. For a software agent, sensors might be API calls that retrieve data from databases or web services.

Percepts: The content that an agent's sensors perceive. A percept is a snapshot of what the agent senses at a particular moment. The complete history of everything an agent has ever perceived is called the percept sequence.

Agent Function: A mathematical function that maps any given percept sequence to an action. This function describes the ideal behavior of the agent - what action it should take for any possible percept sequence.

Agent Program: The concrete implementation of the agent function. While the agent function is an abstract mathematical description, the agent program is the actual code that runs on the agent's hardware.

Actuators: Mechanisms that allow the agent to take actions in the environment. For a robot, these include motors, speakers, and displays. For a software agent, actuators might be functions that modify databases, send messages, or control other systems.

1.2 Rationality and Performance Measures

A **rational agent** is one that does the right thing - but what does "right" mean? We define rationality in terms of a **performance measure** that evaluates the agent's behavior. A rational agent selects actions that maximize the expected value of the performance measure, given what it has perceived so far and any built-in knowledge.

It's important to note that rationality is not the same as omniscience or perfection. A rational agent might make decisions that turn out badly due to factors it couldn't have known about. What matters is that the agent made the best decision possible given the information available at the time. This is called **bounded rationality** - making optimal decisions within the constraints of limited computational resources and incomplete information.

1.3 The PEAS Framework

To design an intelligent agent, we must fully specify the task environment. The PEAS framework provides a structured way to describe this environment:

Component	Description	Example (Self-Driving Car)
Performance	Criteria for success - how we measure the agent's behavior	Safety, reaching destination, comfort, fuel efficiency, obeying traffic laws
Environment	The external world in which the agent operates	Roads, traffic, pedestrians, weather conditions, other vehicles
Actuators	Mechanisms the agent uses to act in the environment	Steering wheel, accelerator, brake, signals, horn
Sensors	Mechanisms the agent uses to perceive the environment	Cameras, LIDAR, radar, GPS, speedometer, engine sensors

SECTION 2: TYPES OF INTELLIGENT AGENTS

Agents can be classified based on their architecture and capabilities. Each type represents an increasingly sophisticated approach to building intelligent systems. Understanding these types helps us choose the appropriate architecture for different problems.

2.1 Simple Reflex Agents

Simple reflex agents select actions based only on the current percept, ignoring the rest of the percept history. They work by finding a rule whose condition matches the current situation (as defined by the percept) and then performing the action associated with that rule. These are called **condition-action rules** or **if-then rules**.

Example: A thermostat is a simple reflex agent. It has one rule: IF temperature < setpoint THEN turn on heater; IF temperature $\geq$ setpoint THEN turn off heater. It doesn't remember past temperatures or predict future ones - it simply reacts to the current temperature.

Limitations: Simple reflex agents only work in fully observable environments where the correct action can be determined from the current percept alone. They cannot handle partially observable environments or situations requiring memory of past events. They also cannot learn or adapt.

2.2 Model-Based Reflex Agents

Model-based reflex agents maintain an **internal state** that keeps track of aspects of the world that are not directly observable. This internal state is updated based on two types of knowledge: (1) how the world evolves independently of the agent, and (2) how the agent's actions affect the world. Together, these constitute the agent's **model** of the world.

Example: A robot vacuum cleaner that remembers which areas it has already cleaned, even when it can't currently see those areas. It maintains a map of the room and updates it as it moves around.

2.3 Goal-Based Agents

Goal-based agents go beyond model-based agents by incorporating **goal information** that describes desirable situations. The agent can reason about which sequences of actions will achieve its goals. This involves consideration of the future - "what will happen if I do such-and-such?" and "will that make me happy?" - which is fundamentally different from the condition-action rules of reflex agents.

Search and Planning: Goal-based agents often use search algorithms (like BFS, DFS, A*) and planning techniques to find sequences of actions that lead from the current state to a goal state. This is where the search problems we studied earlier become directly applicable.

2.4 Utility-Based Agents

While goals provide a binary distinction between "happy" and "unhappy" states, a **utility function** provides a more general measure of happiness. A utility function maps a state (or sequence of states) to a real number representing how desirable that state is. Utility-based agents aim to maximize their expected utility.

Advantages: Utility functions allow agents to (1) make tradeoffs between conflicting goals, (2) handle uncertainty by maximizing expected utility, and (3) compare very different types of outcomes on a common scale. For example, an autonomous vehicle must trade off between reaching the destination quickly (one goal) and minimizing the risk of accidents (another goal).

2.5 Learning Agents

Learning agents can improve their behavior over time based on experience. A learning agent has four conceptual components:

■ COMPONENTS OF A LEARNING AGENT

- 1. Learning Element:** Responsible for making improvements. It uses feedback from the critic to determine how the performance element should be modified to do better in the future.
- 2. Performance Element:** Responsible for selecting external actions. This is what we have previously considered the entire agent - it takes in percepts and decides on actions.
- 3. Critic:** Provides feedback on how the agent is doing with respect to a fixed performance standard. This tells the learning element how well the agent is doing and whether changes are needed.
- 4. Problem Generator:** Responsible for suggesting actions that will lead to new and informative experiences. Without this, the agent might keep doing what it's comfortable with and never discover better approaches.

2.6 Agent Types Comparison

Agent Type	Memory	Goals	Learning	Best For
Simple Reflex	None	None	No	Fully observable, simple tasks
Model-Based	Internal state	None	No	Partially observable environments
Goal-Based	State + Goals	Yes	No	Planning and search problems
Utility-Based	State + Utility	Implicit	No	Decisions with tradeoffs
Learning	All above	Yes	Yes	Unknown or changing environments

SECTION 3: TYPES OF ENVIRONMENTS

The nature of the environment significantly affects the design of the agent. Understanding environment properties helps us choose appropriate algorithms and architectures.

Property	Type A	Type B	Examples
Observability	Fully Observable: Agent sees complete state	Partially Observable: Only partial view	Chess (full) vs Poker (partial)
Determinism	Deterministic: Next state determined by current state + action	Stochastic: Randomness involved	Calculator (det) vs Weather (stoch)
Episodic vs Sequential	Episodic: Each action independent	Sequential: Current action affects future	Image classification (ep) vs Chess (seq)
Static vs Dynamic	Static: Environment doesn't change while agent thinks	Dynamic: Environment changes continuously	Crossword (static) vs Driving (dynamic)
Discrete vs Continuous	Discrete: Finite number of states/actions	Continuous: Infinite states/actions	Chess (discrete) vs Self-driving (cont)
Single vs Multi-Agent	Single: Only one agent	Multi: Multiple interacting agents	Puzzle (single) vs Game (multi)

SECTION 4: KNOWLEDGE REPRESENTATION AND REASONING

4.1 Why Search Algorithms Are Not Enough

While search algorithms like BFS, DFS, and A* are powerful tools for finding paths and solutions, they have significant limitations. Search algorithms work with a **graphical representation** of the state space, but they don't truly "understand" the problem domain. They cannot:

- Represent complex relationships between entities
- Reason about abstract concepts like time, causality, or beliefs
- Make inferences from known facts to derive new knowledge
- Handle incomplete or uncertain information gracefully
- Transfer knowledge from one problem to another

This is where **Knowledge Representation and Reasoning (KRR)** becomes essential. KRR provides formal languages and methods for encoding knowledge about the world in a form that computers can use to solve complex problems and make intelligent decisions.

4.2 What is Knowledge Representation?

Knowledge Representation is a way to encode knowledge about the world in a form that a computer can use to solve complex tasks. It involves choosing appropriate data structures and reasoning methods to represent different types of knowledge - facts, rules, relationships, and procedures.

■ KEY INSIGHT: THE KNOWLEDGE REPRESENTATION HYPOTHESIS

Any mechanically intelligent behavior can be realized by a system that explicitly represents knowledge about its domain and uses that knowledge to reason about its actions. This hypothesis suggests that:

- Intelligence requires the ability to store and manipulate symbolic representations of knowledge
- These representations must capture the essential features of the problem domain
- The system must be able to reason with these representations to solve problems
- Knowledge can be made explicit, examined, modified, and transferred

4.3 Knowledge-Based Agents

A **knowledge-based agent** maintains a **knowledge base (KB)** - a set of sentences expressed in a knowledge representation language. The agent can:

- **TELL:** Add new sentences (facts or rules) to the knowledge base
- **ASK:** Query the knowledge base to determine what is known
- **Derive:** Use inference to derive new knowledge from existing knowledge

The knowledge base starts with **background knowledge** about the domain. As the agent perceives its environment, it TELLS new information to the KB. When deciding what action to take, it ASKS the KB for guidance. The inference engine derives new knowledge from old.

■■ ARCHITECTURE OF A KNOWLEDGE-BASED AGENT

1. Knowledge Base (KB):

- Contains sentences in a formal language (e.g., propositional logic, first-order logic)
- Stores facts about the world (declarative knowledge)
- Stores rules that relate facts (procedural knowledge)

2. Inference Engine:

- Applies logical rules to derive new knowledge from existing knowledge
- Answers queries by checking what can be inferred from the KB
- Must be both SOUND (only derives true conclusions) and COMPLETE (derives all true conclusions)

3. Agent Program Cycle:

- Perceive environment → TELL percepts to KB
- ASK KB what action to take
- Execute action
- TELL KB that action was taken
- Repeat

SECTION 5: TYPES OF KNOWLEDGE REPRESENTATION

5.1 Logical Representation

Logical representation uses formal logic to represent knowledge. The main types are:

Propositional Logic: Uses simple propositions (statements that are either true or false) and logical connectives (AND, OR, NOT, IMPLIES). It's simple but limited in expressiveness - cannot represent relationships between objects or make generalizations.

First-Order Logic (Predicate Logic): Extends propositional logic with objects, relations, and quantifiers (FOR ALL, THERE EXISTS). Much more expressive - can represent complex relationships and make general statements about classes of objects.

5.2 Semantic Networks

Semantic networks represent knowledge as a graph where nodes represent concepts or objects and edges represent relationships between them. Common relationships include IS-A (class membership), HAS-A (attributes), and custom domain-specific relations.

Example: A semantic network for animals might have nodes for "Bird", "Canary", "Robin", with IS-A links from Canary and Robin to Bird, and property links like "can fly", "has wings" associated with Bird.

5.3 Frames

Frames organize knowledge into structures that represent stereotyped situations or objects. Each frame has **slots** (attributes) with **fillers** (values). Frames can inherit properties from parent frames, making them efficient for representing hierarchical knowledge.

5.4 Production Rules (Rule-Based Systems)

Production rules are IF-THEN statements that encode procedural knowledge. The IF part (condition or antecedent) specifies when the rule applies; the THEN part (action or consequent) specifies what to do. Expert systems heavily rely on production rules.

Type	Strengths	Weaknesses	Use Cases
Propositional Logic	Simple, well-understood inference	Limited expressiveness	Simple rule-based systems
First-Order Logic	Very expressive, powerful inference	Complex, can be slow	Theorem proving, databases
Semantic Networks	Intuitive, visual representation	Informal semantics	Concept hierarchies, NLP
Frames	Structured, supports defaults	Less flexible than logic	Object-oriented domains
Production Rules	Natural for procedures	Can be hard to maintain	Expert systems

SECTION 6: TYPES OF REASONING

Reasoning is the process of drawing conclusions from available knowledge. Different types of reasoning are used depending on the nature of the problem and the available information.

6.1 Deductive Reasoning (General → Specific)

Deductive reasoning draws specific conclusions from general premises. If the premises are true and the reasoning is valid, the conclusion is guaranteed to be true. This is the foundation of formal logic.

EXAMPLE OF DEDUCTIVE REASONING

Premise 1 (General Rule): All birds can fly.

Premise 2 (Specific Fact): Tweety is a bird.

Conclusion: Therefore, Tweety can fly.

This conclusion is **logically certain** if the premises are true. However, note that the premises might be false (e.g., penguins are birds but cannot fly), which illustrates the difference between valid reasoning and sound reasoning.

6.2 Inductive Reasoning (Specific → General)

Inductive reasoning draws general conclusions from specific observations. Unlike deduction, inductive conclusions are not guaranteed to be true - they are probable based on the evidence. Machine learning is essentially automated inductive reasoning.

EXAMPLE OF INDUCTIVE REASONING

Observation 1: Swan 1 is white.

Observation 2: Swan 2 is white.

Observation 3: Swan 3 is white.

... (many more observations of white swans)

Conclusion: All swans are white.

This conclusion is **probable but not certain**. (In fact, black swans do exist!) Inductive reasoning is how we learn general patterns from specific examples - the foundation of machine learning.

6.3 Abductive Reasoning (Best Explanation)

Abductive reasoning (also called inference to the best explanation) starts with observations and finds the most likely explanation. It's commonly used in medical diagnosis, debugging, and detective work.

EXAMPLE OF ABDUCTIVE REASONING

Observation: The patient has fever and rash.

Known Rule: Dengue causes fever and rash.

Abductive Conclusion: The patient probably has dengue.

This is finding the **best explanation** for the observations. It's not certain (other diseases could cause these symptoms), but it's a reasonable hypothesis that can be tested further.

Reasoning Type	Direction	Certainty	Used In
Deductive	General → Specific	Certain (if premises true)	Theorem proving, logic programming
Inductive	Specific → General	Probable (not guaranteed)	Machine learning, pattern recognition
Abductive	Effect → Cause	Plausible (best guess)	Diagnosis, debugging, explanation

SECTION 7: SAMPLE QUIZ QUESTIONS

7.1 Multiple Choice Questions

1. Which type of agent maintains an internal model of the world?

- a) Simple Reflex Agent
- b) Model-Based Reflex Agent
- c) Goal-Based Agent
- d) None of the above

Answer: b) Model-Based Reflex Agent

2. In the PEAS framework, 'E' stands for:

- a) Efficiency
- b) Evaluation
- c) Environment
- d) Execution

Answer: c) Environment

3. Which reasoning type is used by machine learning?

- a) Deductive
- b) Inductive
- c) Abductive
- d) None

Answer: b) Inductive (Specific → General patterns)

4. A knowledge-based agent uses which operations on its KB?

- a) READ and WRITE
- b) TELL and ASK
- c) INPUT and OUTPUT
- d) PUSH and POP

Answer: b) TELL and ASK

5. Which environment type has outcomes involving randomness?

- a) Deterministic
- b) Episodic
- c) Stochastic
- d) Static

Answer: c) Stochastic

6. First-Order Logic extends Propositional Logic with:

- a) Neural networks
- b) Objects, relations, and quantifiers
- c) Probability
- d) Search algorithms

Answer: b) Objects, relations, and quantifiers

7. Which agent type can improve its performance over time?

- a) Simple Reflex
- b) Model-Based
- c) Goal-Based
- d) Learning Agent

Answer: d) Learning Agent

8. An inference engine must be both:

- a) Fast and Slow

- b) Sound and Complete
- c) Simple and Complex
- d) Static and Dynamic

Answer: b) Sound (correct) and Complete (finds all truths)

7.2 True/False Questions

1. A rational agent always makes perfect decisions. **(FALSE - Rational agents make optimal decisions given available information, not perfect ones)**
2. Simple reflex agents can work in partially observable environments. **(FALSE - They require fully observable environments)**
3. Deductive reasoning can produce conclusions that are merely probable. **(FALSE - Deductive conclusions are certain if premises are true)**
4. Semantic networks represent knowledge as graphs. **(TRUE)**
5. The TELL operation adds new knowledge to the knowledge base. **(TRUE)**
6. Utility functions provide binary (yes/no) measures of desirability. **(FALSE - They provide continuous numerical measures)**
7. Abductive reasoning finds the best explanation for observations. **(TRUE)**
8. Model-based agents have no memory of past states. **(FALSE - They maintain internal state)**

SECTION 8: KEY CONCEPTS SUMMARY

KEY FORMULAS AND CONCEPTS TO REMEMBER

Agent = Sensors + Agent Function + Actuators

PEAS Framework: Performance, Environment, Actuators, Sensors

Agent Types (in order of complexity):

Simple Reflex → Model-Based → Goal-Based → Utility-Based → Learning

Knowledge-Based Agent Operations:

- TELL: Add knowledge to KB
- ASK: Query the KB
- Inference: Derive new knowledge

Reasoning Types:

- Deductive: General → Specific (certain conclusions)
- Inductive: Specific → General (probable conclusions)
- Abductive: Effect → Cause (best explanation)

Inference Requirements:

- Sound: Only derives true conclusions from true premises
- Complete: Derives all conclusions that follow from premises

Environment Properties:

Observable (Full/Partial) | Deterministic/Stochastic | Episodic/Sequential
Static/Dynamic | Discrete/Continuous | Single/Multi-Agent

— *End of Revision Note 2* —
Best of luck with your exam preparation!